creator studio page

✅ Stworzyłem `Centrum Kreacji` `/dashboard/studio`:
- 🎛️ Sekcja edycji nakładki Live Feed (OBS)- 💸 Sekcja konfiguracji Widgeta + QR + PDF + embed script
Masz teraz centralne miejsce do personalizacji i promocji twórcy.
// app/dashboard/studio/page.tsx'use client';
import dynamic from 'next/dynamic';import { useSession } from 'next-auth/react';import OverlayEditor from '@/components/OverlayEditor';import WidgetPreview from '@/components/WidgetPreview';
const CreatorStudioPage = () => {const { data: session } = useSession();const handle = session?.user?.username || session?.user?.id || 'me';
return (<main className="max-w-6xl mx-auto px-4 py-10 space-y-16"><h1 className="text-3xl font-bold mb-6">🎨 Centrum Kreacji</h1>
``` <section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"> <h2 className="text-xl font-semibold">📺 Nakładka Live Feed (OBS)</h2> <OverlayEditor /> </div> <iframe src={`/overlay?creatorId=${handle}&test=true`} className="w-full h-80 rounded border shadow" title="Podgląd nakładki" /> </section>
<section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"> <h2 className="text-xl font-semibold">💸 Widget Napiwków + QR</h2> <WidgetPreview handle={handle} /> </div> <iframe src={`/widget/preview?handle=${handle}`} className="w-full h-72 rounded border shadow" title="Podgląd widgetu" /> </section></main>
```
);};
export default CreatorStudioPage;
[page.tsx](https://www.notion.so/page-tsx-221120af4489805a9f74fc0d706589a0?pvs=21)
[widget previev](https://www.notion.so/widget-previev-221120af448980c09b2eda01823eb0ef?pvs=21)
[widget true](https://www.notion.so/widget-true-221120af44898011a29cdbb4b39885b0?pvs=21)
[widget store ](https://www.notion.so/widget-store-221120af448980ccb163f9b11d3e7cf3?pvs=21)
[`Hover` QuickSlider Widget ](https://www.notion.so/Hover-QuickSlider-Widget-221120af4489805ba52fd473b8f89714?pvs=21)
[widget loader](https://www.notion.so/widget-loader-221120af4489807db797f8e82515e934?pvs=21)
[QuickSlider Widget](https://www.notion.so/QuickSlider-Widget-221120af44898084a400ecfc7907e51a?pvs=21)

# page.tsx

// app/dashboard/studio/page.tsx'use client';
import dynamic from 'next/dynamic';import { useSession } from 'next-auth/react';import OverlayEditor from '@/components/OverlayEditor';import WidgetPreview from '@/components/WidgetPreview';
const CreatorStudioPage = () => {const { data: session } = useSession();const handle = session?.user?.username || session?.user?.id || 'me';
return (<main className="max-w-6xl mx-auto px-4 py-10 space-y-16"><h1 className="text-3xl font-bold mb-6">🎨 Centrum Kreacji</h1>
``` <section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"> <h2 className="text-xl font-semibold">📺 Nakładka Live Feed (OBS)</h2> <OverlayEditor /> </div> <iframe src={`/overlay?creatorId=${handle}&test=true`} className="w-full h-80 rounded border shadow" title="Podgląd nakładki" /> </section>
<section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"> <h2 className="text-xl font-semibold">💸 Widget Napiwków + QR</h2> <WidgetPreview handle={handle} /> </div> <iframe src={`/widget/preview?handle=${handle}`} className="w-full h-72 rounded border shadow" title="Podgląd widgetu" /> </section></main>
```
);};
export default CreatorStudioPage;

# widget previev
// app/widget/preview/page.tsx'use client';
import { useSearchParams } from 'next/navigation';import { useEffect, useState } from 'react';
export default function WidgetPreviewPage() {const searchParams = useSearchParams();const handle = searchParams.get('handle') || 'me';const [open, setOpen] = useState(false);
const click = () => setOpen((o) => !o);
useEffect(() => {window.addEventListener('message', (e) => {if (e.data === 'toggleModal') click();});}, []);
return ( <buttononClick={click}className="px-4 py-2 rounded text-white"style={{ backgroundColor: '#006D6D' }}>💸 Wesprzyj {handle}</button>
``` {open && ( <div className="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-50"> <div className="bg-white p-6 rounded-lg max-w-sm w-full relative"> <button onClick={click} className="absolute top-2 right-3 text-gray-600 hover:text-black" > ✖ </button> <h2 className="text-xl font-bold mb-2">🎁 Wesprzyj twórcę @{handle}</h2> <input type="number" placeholder="Kwota (USDC)" className="w-full border px-3 py-2 rounded mb-3" /> <textarea placeholder="Wiadomość (opcjonalna)" className="w-full border px-3 py-2 rounded mb-4" rows={2} /> <button className="bg-[#FFD700] text-black px-4 py-2 rounded w-full font-semibold"> Tip It 💛 </button> </div> </div> )}</div>
```

);}

widget true
🔥 **Plan: TipWidget z pełną personalizacją**
1. Struktura osadzalna
```htmlhtmlKopiujEdytuj<script src="https://tipjar.plus/widget.js"
data-creator="janKowalski"></script>
```
2. Twórca konfiguruje:
- Rozmiar: `small`, `medium`, `large`- Kształt: `circle`, `rounded`, `square`- Kolory: tło, tekst, hover- Ikona: własny `.png`/`.svg` lub gotowy preset (np. 💸, 🎁, ☕)- Domyślny tekst: “Wesprzyj mnie”, “Tip me”, “Buy me a kebab”- Gdzie pojawić się slider (on hover, on click, always open)- Zachowanie: modal czy redirect do profilu
3. Po kliknięciu:
- Rozwija się (w tej samej ramce) **mini-slider tipa** z: - szybkie kwoty (5/10/25 USDC) - wiadomość - input na kwotę - `Tip Now` z przekierowaniem lub embedowanym flow
---
💻 Techniczna realizacja
- Widget to `iframe` lub ShadowDOM → brak kolizji CSS/JS- Komunikacja przez `postMessage()` do otwarcia modala/redirectu- Frontend hostowany na `tipjar.plus/widget`

✅ Etapy wdrożenia
1. `tipjar.plus/widget.js` loader skrypt2. `WidgetConfig` API per creator (`/api/widget/settings/:id`)3. `WidgetModal` z `QuickTipForm`4. Panel edycji w dashboardzie `/dashboard/widget`5. Generator kodu + podgląd live + QR do wklejenia
---
📦 Mogę zacząć od Zustand store + typów konfiguracji + komponentu `WidgetBubble`.
// public/widget.js(function () {const script = document.currentScript;const creator = script.get zaAttribute('data-creator') || 'me';const iframe = document.createElement('iframe'); iframe.src = `https://tipjar.plus/widget/preview?handle=${creator}`;iframe.style.border = 'none';iframe.style.width = '200px';iframe.style.height = '60px';iframe.style.position = 'fixed';iframe.style.bottom = '20px';iframe.style.right = '20px';iframe.style.zIndex = '9999';iframe.allowTransparency = true; document.body.appendChild(iframe);})();

widget store
// lib/stores/widgetStore.tsimport { create } from 'zustand';
export interface TipWidgetConfig {shape: 'circle' | 'rounded' | 'square';size: 'small' | 'medium' | 'large';bgColor: string;textColor: string;iconType: 'custom' | 'emoji' | 'preset';iconValue: string; // URL lub emojilabel: string; // np. "Tip Me"behavior: 'modal' | 'redirect';openStyle: 'click' | 'hover' | 'always';}
interface TipWidgetStore {config: TipWidgetConfig;setConfig: (conf: Partial<TipWidgetConfig>) => void;resetConfig: () => void;}

const defaultConfig: TipWidgetConfig = {shape: 'rounded',size: 'medium',bgColor: '#006D6D',textColor: '#FFFFFF',iconType: 'emoji',iconValue: '💸',label: 'Tip Me',behavior: 'modal',openStyle: 'click',};
export const useWidgetStore = create<TipWidgetStore>((set) => ({config: defaultConfig,setConfig: (conf) =>set((state) => ({config: {...state.config,...conf,},})),resetConfig: () => set({ config: defaultConfig }),}));

`Hover` QuickSlider Widget
✅ Zachowanie
1. **Początkowo widoczny tylko okrągły przycisk 💸**2. **Po najechaniu**: - Rozsuwa się poziomy suwak `0 – 20` USDC - Bez cyfr – tylko cienki pasek z kropką3. **Po kliknięciu suwaka**: - Pojawia się mini-karta (`popup modal`) - Pokazuje kwotę, awatar, `Tip Now`

🔧 Architektura
- Komponent `HoverSliderWidget.tsx`- Komunikacja z modalkiem przez Zustand lub `postMessage`- Karta wyświetlana inline – bez `alert` ani `window.open`

🎯 Etapy
1. `HoverSliderWidget.tsx`2. Callback: klik = `setTipAmount(x)` → `openModal()`3. Mini modal z `amount`, `avatar`, `@handle`4. Podgląd w `/widget/preview?handle=...&style=slider`

Chcesz teraz `HoverSliderWidget` z `range` + trigger modala?
// app/dashboard/studio/page.tsx'use client';
import dynamic from 'next/dynamic';import { useSession } from 'next-auth/react';import OverlayEditor from '@/components/OverlayEditor';import WidgetPreview from '@/components/WidgetPreview';import { useState } from 'react';
const CreatorStudioPage = () => {const { data: session } = useSession();const handle = session?.user?.username || session?.user?.id || 'me';const [style, setStyle] = useState<'button' | 'slider'>('button');
return (<main className="max-w-6xl mx-auto px-4 py-10 space-y-16"><h1 className="text-3xl font-bold mb-6">🎨 Centrum Kreacji</h1>
``` <section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"> <h2 className="text-xl font-semibold">📺 Nakładka Live Feed (OBS)</h2> <OverlayEditor /> </div> <iframe src={`/overlay?creatorId=${handle}&test=true`} className="w-full h-80 rounded border shadow" title="Podgląd nakładki" /> </section>
 <section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"> <h2 className="text-xl font-semibold">💸 Widget Napiwków + QR</h2> <div className="flex gap-3 items-center text-sm"> <label className="font-semibold">Styl:</label> <select value={style} onChange={(e) => setStyle(e.target.value as 'button' | 'slider')} className="border px-2 py-1 rounded" > <option value="button">🟨 Przycisk</option> <option value="slider">🎚️ Suwak</option> </select> </div> <WidgetPreview handle={handle} /> </div>
```

<iframe src={`/widget/preview?handle=${handle}&style=${style}`} className="w-full h-72 rounded border shadow" title="Podgląd widgetu" /> </section></main>
```
);};
export default CreatorStudioPage;

# widget loader
✅ `widget.js` gotowy – skrypt do osadzania na dowolnej stronie:
- Tworzy `iframe` z osadzonym widgetem TipJar+- Ładuje `https://tipjar.plus/widget/preview?handle=...`- Osadza się w prawym dolnym rogu, z wysokim z-index
---
Przykładowe użycie na stronie twórcy:
```htmlhtmlKopiujEdytuj<script src="https://tipjar.plus/widget.js" data-creator="janKowalski"></script>
```
Co dalej? Animacja przy pojawieniu? Obsługa niestandardowych stylów z widgetStore?
// public/widget.js(function () {const script = document.currentScript;const creator = script.getAttribute('data-creator') || 'me';const iframe = document.createElement('iframe'); iframe.src = `https://tipjar.plus/widget/preview?handle=${creator}`;iframe.style.border = 'none';iframe.style.width = '200px';iframe.style.height = '60px';iframe.style.position = 'fixed';iframe.style.bottom = '20px';iframe.style.right = '20px';iframe.style.zIndex = '9999';iframe.allowTransparency = true; document.body.appendChild(iframe);})();

QuickSlider Widget

Zachowanie
1. **Początkowo widoczny tylko okrągły przycisk 💸**2. **Po najechaniu**: - Rozsuwa się poziomy suwak `0 – 20` USDC - Bez cyfr – tylko cienki pasek z kropką3. **Po kliknięciu suwaka**: - Pojawia się mini-karta (`popup modal`) - Pokazuje kwotę, awatar, `Tip Now`

🔧 Architektura
- Komponent `HoverSliderWidget.tsx`- Komunikacja z modalkiem przez Zustand lub `postMessage`- Karta wyświetlana inline – bez `alert` ani `window.open`

🎯 Etapy
1. `HoverSliderWidget.tsx`2. Callback: klik = `setTipAmount(x)` → `openModal()`3. Mini modal z `amount`, `avatar`, `@handle`4. Podgląd w `/widget/preview?handle=...&style=slider`
// app/dashboard/studio/page.tsx'use client';
import dynamic from 'next/dynamic';import { useSession } from 'next-auth/react';import OverlayEditor from '@/components/OverlayEditor';import WidgetPreview from '@/components/WidgetPreview';import { useState } from 'react';
```

const CreatorStudioPage = () => {const { data: session } = useSession();const handle = session?.user?.username || session?.user?.id || 'me';const [style, setStyle] = useState<'button' | 'slider'>('button');
return (<main className="max-w-6xl mx-auto px-4 py-10 space-y-16"><h1 className="text-3xl font-bold mb-6">🎨 Centrum Kreacji</h1>
``` <section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"> <h2 className="text-xl font-semibold">📺 Nakładka Live Feed (OBS)</h2> <OverlayEditor /> </div> <iframe src={`/overlay?creatorId=${handle}&test=true`} className="w-full h-80 rounded border shadow" title="Podgląd nakładki" /> </section>
  <section className="grid md:grid-cols-2 gap-8 items-start"> <div className="space-y-4"><h2 className="text-xl font-semibold">💸 Widget Napiwków + QR</h2> <div className="flex gap-3 items-center text-sm"> <label className="font-semibold">Styl:</label> <select value={style} onChange={(e) => setStyle(e.target.value as 'button' | 'slider')} className="border px-2 py-1 rounded" > <option value="button">🟨 Przycisk</option> <option value="slider">🎚️ Suwak</option> </select> </div> <WidgetPreview handle={handle} /> </div> <iframe src={`/widget/preview?handle=${handle}&style=${style}`} className="w-full h-72 rounded border shadow" title="Podgląd widgetu" /> </section></main>
```
);};
export default CreatorStudioPage;

# overlay settings
✅ API backendowe `/overlay/settings/:creatorId` gotowe:
- `GET` — pobiera ustawienia nakładki- `POST` — zapisuje zmiany (autoryzacja JWT)
---
Kolejny krok: `OverlaySettingsService.ts` + model Prisma (jeśli nie istnieje). Gotowy? → `NEXT`.
### Powiedziałeś(-aś):
Next
### ChatGPT powiedział:
Edytowano Overlay-settings
✅ `OverlaySettingsService` gotowy – obsługuje:
- `get()` z Prisma `findUnique`- `save()` z `upsert` (update lub create)
---
Ostatni krok backendu: dodanie modelu Prisma `overlaySettings` + migracja.
// app/overlay/page.tsx (transparent-ready)'use client';
import { useEffect, useState, useRef } from 'react';import { AnimatePresence, motion } from 'framer-motion';import io from 'socket.io-client';
interface TipEntry {id: string;nickname: string;avatarUrl?: string;amount: number;message?: string;}
export default function OverlayPage({ searchParams }: { searchParams: { creatorId: string;

test?: string } }) {const [queue, setQueue] = useState<TipEntry[]>([]);const [current, setCurrent] = useState<TipEntry | null>(null);const { creatorId, test } = searchParams;const socket = useRef<any>(null);
useEffect(() => {if (!creatorId) return;
```if (test === 'true') {  const interval = setInterval(() => {    setQueue((q) => [      ...q,      {        id: Date.now().toString(),        nickname: 'TestFan',        amount: Math.floor(Math.random() * 200), message: 'This is a demo tip! 💸',      },    ]);  }, 5000);  return () => clearInterval(interval);}
socket.current = io('wss://tipjar.plus', {  query: { creatorId },});
socket.current.on('tip', (data: TipEntry) => {  setQueue((q) => [...q, data]);});
return () => socket.current?.disconnect();
```
}, [creatorId, test]);
useEffect(() => {if (!current && queue.length > 0) {const [first, ...rest] = queue;setCurrent(first);setQueue(rest);const timeout = setTimeout(() => setCurrent(null), 8000);return () => clearTimeout(timeout);}}, [current, queue]);
return (<divclassName="fixed inset-0 z-50 flex items-end justify-start p-4 pointer-events-none"style={{ backgroundColor: 'rgba(0,0,0,0)' }} // transparent for OBS><AnimatePresence>{current && (<motion.divkey={current.id}initial={{ opacity: 0, y: 30 }}animate={{ opacity: 1, y: 0 }}exit={{ opacity: 0, y: -20 }}transition={{ duration: 0.5 }}className="bg-[#0f0f0fcc] backdrop-blur text-white rounded-lg p-4 max-w-sm shadow-lg"> {current.nickname} +{current.amount.toFixed(2)} USDC {current.message && {current.message} }</motion.div>)}</AnimatePresence>);}
Oto model Prisma do tabeli `overlaySettings`:

📁 `prisma/schema.prisma`

Live Feed Overlay Tip Jar
// app/overlay/[creatorId]/page.tsx'use client';
import { useEffect, useState, useRef } from 'react';import { AnimatePresence, motion } from 'framer-motion';import io, { Socket } from 'socket.io-client';
interface TipEntry {id: string;nickname: string;avatarUrl?: string;amount: number;message?: string;}
export default function LiveTipOverlay({ searchParams }: { searchParams: { creatorId: string, test?: string } }) {const [queue, setQueue] = useState<TipEntry[]>([]);const [current, setCurrent] = useState<TipEntry | null>(null);const socket = useRef<Socket | null>(null);const { creatorId, test } = searchParams;
// Polling fallback in case WebSocket failsuseEffect(() => {let pollInterval: NodeJS.Timeout;
```if (!test && !socket.current && creatorId) { pollInterval = setInterval(async () => {   try {   const res = await fetch(`/api/tips/stream/${creatorId}`);   if (res.ok) {   const tip: TipEntry = await res.json();   if (tip) {   setQueue((prev) => [...prev, tip]);   }   }   } catch (e) {
```

```
console.error('Polling error:', e);    }  }, 7000);}return () => clearInterval(pollInterval);
```
}, [creatorId, test]);
useEffect(() => {if (test === 'true') {const interval = setInterval(() => {const fakeTip: TipEntry = {id: Date.now().toString(),nickname: 'TestFan',amount: Math.floor(Math.random() * 100),message: 'This is a test tip!',};setQueue((prev) => [...prev, fakeTip]);}, 5000);return () => clearInterval(interval);}
```socket.current = io('wss://tipjar.plus', {  query: { creatorId },  transports: ['websocket'],});
socket.current.on('tip', (tip: TipEntry) => {  setQueue((prev) => [...prev, tip]);});
return () => {  socket.current?.disconnect();};
```
}, [creatorId, test]);
useEffect(() => {if (!current && queue.length > 0) {const [next, ...rest] = queue;setCurrent(next);setQueue(rest);const timeout = setTimeout(() => setCurrent(null), 7000);return () => clearTimeout(timeout);}}, [current, queue]);
return (<AnimatePresence>{current && (<motion.divkey={current.id}initial={{ opacity: 0, y: 40 }}animate={{ opacity: 1, y: 0 }}exit={{ opacity: 0, y: -20 }}transition={{ duration: 0.4 }}className="flex items-start gap-3">{current.avatarUrl && (<imgsrc={current.avatarUrl}alt={current.nickname}className="w-12 h-12 rounded-full border-2 border-[#FFD700]"/>)} <h4 className="font-display font-semibold text-[#FFD700] text-lg">{current.nickname}</h4> <imgsrc="https://cryptologos.cc/logos/usd-coin-usdc-logo.png"alt="USDC"className="w-5 h-5"/>+{current.amount.toFixed(2)} USDC {current.message && ({current.message})}
```        {current.amount >= 50 && (          <motion.div            className="absolute -top-10 -left-10 w-[400px] h-[400px] pointer-events-none z-[-1]"            initial={{ opacity: 0 }}            animate={{ opacity: 1 }}            exit={{ opacity: 0 }}            transition={{ duration: 1 }}          >
<div className="w-full h-full animate-ping-slow rounded-full bg-[#FFD700]/30" />
</motion.div>        )}
    </motion.div>    )}  </AnimatePresence></div>
```
);}

TipWidget
Plan: TipWidget z pełną personalizacją
1. Struktura osadzalna
```htmlhtmlKopiujEdytuj<script src="https://tipjar.plus/widget.js" data-creator="janKowalski"></script>
```
2. Twórca konfiguruje:

- Rozmiar: `small`, `medium`, `large`- Kształt: `circle`, `rounded`, `square`- Kolory: tło, tekst, hover- Ikona: własny `.png`/`.svg` lub gotowy preset (np. 💸, 🎁, ☕)- Domyślny tekst: “Wesprzyj mnie”, “Tip me”, “Buy me a kebab”- Gdzie pojawić się slider (on hover, on click, always open)- Zachowanie: modal czy redirect do profilu
3. Po kliknięciu:
- Rozwija się (w tej samej ramce) **mini-slider tipa** z: - szybkie kwoty (5/10/25 USDC) - wiadomość - input na kwotę - `Tip Now` z przekierowaniem lub embedowanym flow
---
💻 Techniczna realizacja
- Widget to `iframe` lub ShadowDOM → brak kolizji CSS/JS- Komunikacja przez `postMessage()` do otwarcia modala/redirectu- Frontend hostowany na `tipjar.plus/widget`

✅ Etapy wdrożenia
1. `tipjar.plus/widget.js` loader skrypt2. `WidgetConfig` API per creator (`/api/widget/settings/:id`)3. `WidgetModal` z `QuickTipForm`4. Panel edycji w dashboardzie `/dashboard/widget`5. Generator kodu + podgląd live + QR do wklejenia
---
📦 Mogę zacząć od Zustand store + typów konfiguracji + komponentu `WidgetBubble`.
[widgetStore.ts](https://www.notion.so/widgetStore-ts-221120af4489807fb9caeaa128e227a2?pvs=21)
[WidgetPreview.tsx](https://www.notion.so/WidgetPreview-tsx-221120af4489806c9bd2d1d471c72e6f?pvs=21)
[QR w **widget preview**
](https://www.notion.so/QR-w-widget-preview-221120af44898051abedf57c5a994066?pvs=21)
[qr headler](https://www.notion.so/qr-headler-221120af44898019a20be3958bcdb411?pvs=21)

WidgetPreview.tsx
✅ Gotowy `WidgetPreview.tsx` – umożliwia:
- Generację linku `tipjar.plus/@ja`- Pobranie QR jako PNG + A4 PDF z linkiem- Edycję kolorów QR- Podgląd widgetu z aktualnymi ustawieniami- Skrypt do wklejenia na stronę
Następny krok: komponent `WidgetPreview.tsx` z opcją:
- podglądu,- kopiowania kodu `<script>`,- generowania QR z kolorem tła/frontu,- eksportem wersji PDF na A4 z linkiem i kodem
// components/WidgetPreview.tsx'use client';
import { useWidgetStore } from '@/lib/stores/widgetStore';import QRCode from 'qrcode.react';import { useEffect, useRef, useState } from 'react';import html2canvas from 'html2canvas';import jsPDF from 'jspdf';
export default function WidgetPreview({ handle }: { handle: string }) {const { config } = useWidgetStore();const containerRef = useRef<HTMLDivElement>(null);const [bg, setBg] = useState('#ffffff');const [fg, setFg] = useState('#000000');const profileUrl = `https://tipjar.plus/@${handle}`;
const scriptTag = `<script src='<https://tipjar.plus/widget.js>' data-creator='${handle}'></script>`;
const downloadQR = () => {const canvas = document.querySelector('canvas') as

HTMLCanvasElement;const url = canvas.toDataURL();const a = document.createElement('a');a.href = url;a.download = `${handle}_qr.png`;a.click();};
const downloadPDF = async () => {if (!containerRef.current) return;const canvas = await html2canvas(containerRef.current);const imgData = canvas.toDataURL('image/png');const pdf = new jsPDF({ orientation: 'portrait', unit: 'px', format: 'a4' });pdf.text(profileUrl, 40, 40);pdf.addImage(imgData, 'PNG', 40, 60, 300, 300);pdf.save(`${handle}_qr_promo.pdf`);};
return (🔗 Link profilu: <code>{profileUrl}</code> Kod osadzenia:<pre className="overflow-x-auto mt-1">{scriptTag}</pre>
```  <div className="flex flex-col md:flex-row gap-6 items-start">  <div className="text-center space-y-2">  <QRCode value={profileUrl} size={192} fgColor={fg} bgColor={bg} />  <div className="flex gap-2 items-center justify-center">  <input type="color" value={bg} onChange={(e) => setBg(e.target.value)} title="Tło" />  <input type="color" value={fg} onChange={(e) => setFg(e.target.value)} title="Kolor kodu" />  </div>  <button onClick={downloadQR} className="bg-blue-100 hover:bg-blue-200 px-4 py-1 rounded" > 📥 Pobierz PNG QR </button> <button onClick={downloadPDF} className="bg-green-100 hover:bg-green-200 px-4 py-1 rounded" > 🖨️ PDF do druku (A4) </button>  </div>
  <div className="p-4 border rounded-md" style={{ backgroundColor: config.bgColor, color: config.textColor }}>  <div style={{ borderRadius: config.shape === 'circle' ? '9999px' : config.shape === 'rounded' ? '12px' : '0px', padding: config.size === 'small' ? 8 : config.size === 'large' ? 20 : 12, fontSize: config.size === 'small' ? 12 : config.size === 'large' ? 20 : 16, display: 'inline-flex', alignItems: 'center', gap: 8, cursor: 'pointer', }} > {config.iconType === 'emoji' ? config.iconValue : <img src={config.iconValue} alt="icon" width={24} height={24} />}
<span>{config.label}</span>  </div>  </div>  </div></div>
```
);}

[creator studio page](#%20creator%20studio%20page%20233120af448980f999d7f15af329f8aa/creator%20studio%20page%20221120af448980288772d19d6799e262.md)